



UNIVERSIDADE FEDERAL
DO ESPÍRITO SANTO

Centro Tecnológico
Departamento de Informática

Prof. Vítor E. Silva Souza

<http://www.inf.ufes.br/~vitorsouza>

Automação de Build com Maven



This material is licensed under the Creative Commons license Attribution-ShareAlike 4.0 International: <http://creativecommons.org/licenses/by-sa/4.0/>.

Crédito

Parte dos slides
deste curso foram
baseados em
material cedido
pelo prof. João
Paulo A. Almeida.



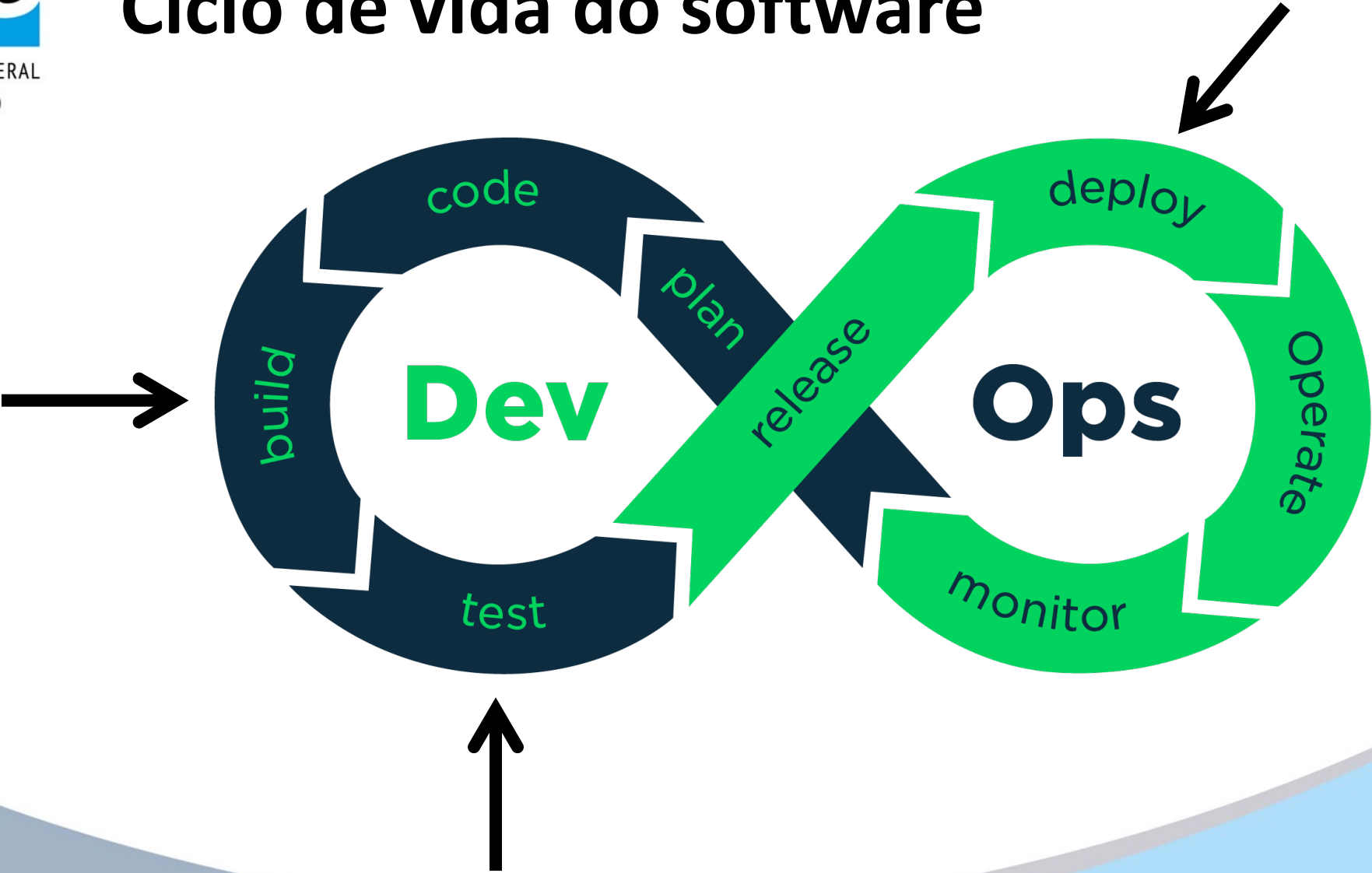
Motivação

- Desenvolver software inclui tarefas tediosas (compilar, testar, empacotar, etc.) se feitas manualmente;
- Trabalhar com grandes equipes sem padronização gera confusão e retrabalho;
- Reuso de código pode causar “*dependency hell*”.

Automação de build

- *Build Automation* ou Automação de Compilação ([Wikipedia](#));
- Criação de scripts para automatizar grande variedade de tarefas que devs realizam no dia-a-dia, incluindo:
 - *Compilar (compile)*;
 - *Testar (test)*;
 - *Empacotar (package)*;
 - *Implantar (deploy)*;
 - *Documentar (document)*.

Ciclo de vida do software



Ferramentas de automação de build



MSBuild



GNU Make



CMake



ruby/rake

A make-like build utility for Ruby.



170 Contributors 3m Used by 6 Discussions 2k Stars 608 Forks



Gradle



Exemplo com Make



```
all: main

main: main.c circulo.h circulo.o
    gcc -o main main.c circulo.o

circulo.o: circulo.h circulo.c
    gcc -c circulo.c

clean:
    @rm -f *.o main
```

```
vitor@localhost Live % make
vitor@localhost Live % ./main
vitor@localhost Live % make clean
```

Exemplo com Ant

 build.xml

```
<project name="TrabProg00" default="compile" basedir=".">
  <property name="src" location="src" />
  <property name="bin" location="bin" />
  <property name="mainClass" value="br.ufes.inf.poo.App" />
  <target name="init" description="Inicializa estruturas.">
    <mkdir dir="${bin}" />
  </target>
  <target name="compile" depends="init" description="Compila.">
    <javac srcdir="${src}" destdir="${bin}" />
  </target>

  ... outros alvos: run, clean, etc.

</project>
```

```
vitor@localhost Live % ant compile
vitor@localhost Live % ant run
vitor@localhost Live % ant clean
```



(Primariamente)

Apache Maven



- Nasceu para tentar simplificar os processos de build de um outro projeto da Apache (Jakarta Turbine);
 - *Vários projetos, cada um com seu build.xml do Ant;*
 - *Arquivos de build levemente diferentes uns dos outros;*
 - *JARs de dependências eram enviados ao repositório CVS.*
- Motivação:
 - *Padronizar a forma de fazer o build dos projetos;*
 - *Facilitar a publicação de informações dos projetos;*
 - *Compartilhar os JARs pelos vários projetos.*

“I just knew there had to be a better way. But to start with I know I wanted: a model for a project so you could look in one place for everything that pertained to the project; a standard directory structure so you didn't have to go fishing around for libraries, sources and documentation. [...] I wanted to make an opinionated piece of software and I preferred the notion of convention over configuration. I wanted a project's infrastructure to look the same and work the same”



Jason van Zyl
Criador do Maven

<https://maven.apache.org/background/history-of-maven.html>

Padronização

- *“On a very high level all projects need to be built, tested, packaged, documented and deployed.”*
- *“Of course there is infinite variation in each of the above mentioned steps, but this variation still occurs within the confines of a well defined path and it is this path that Maven attempts to present to everyone in a clear way”*
- *“Maven was born of the very practical desire to make several projects at Apache work in the same way. So that developers could freely move between these projects, knowing clearly how they all worked by understanding how one of them worked. lear way.”*

<https://maven.apache.org/background/philosophy-of-maven.html>

Dependências declarativas



The next thing I noticed were all the JARs that we were depending on were stored in CVS. We had many copies of Xerces laying around which is a waste of space, every time the version of Xerces changed I had to update the copies of Xerces in each of the projects, but more importantly without some declarative statement of your dependencies there is no way you could perform any analysis.

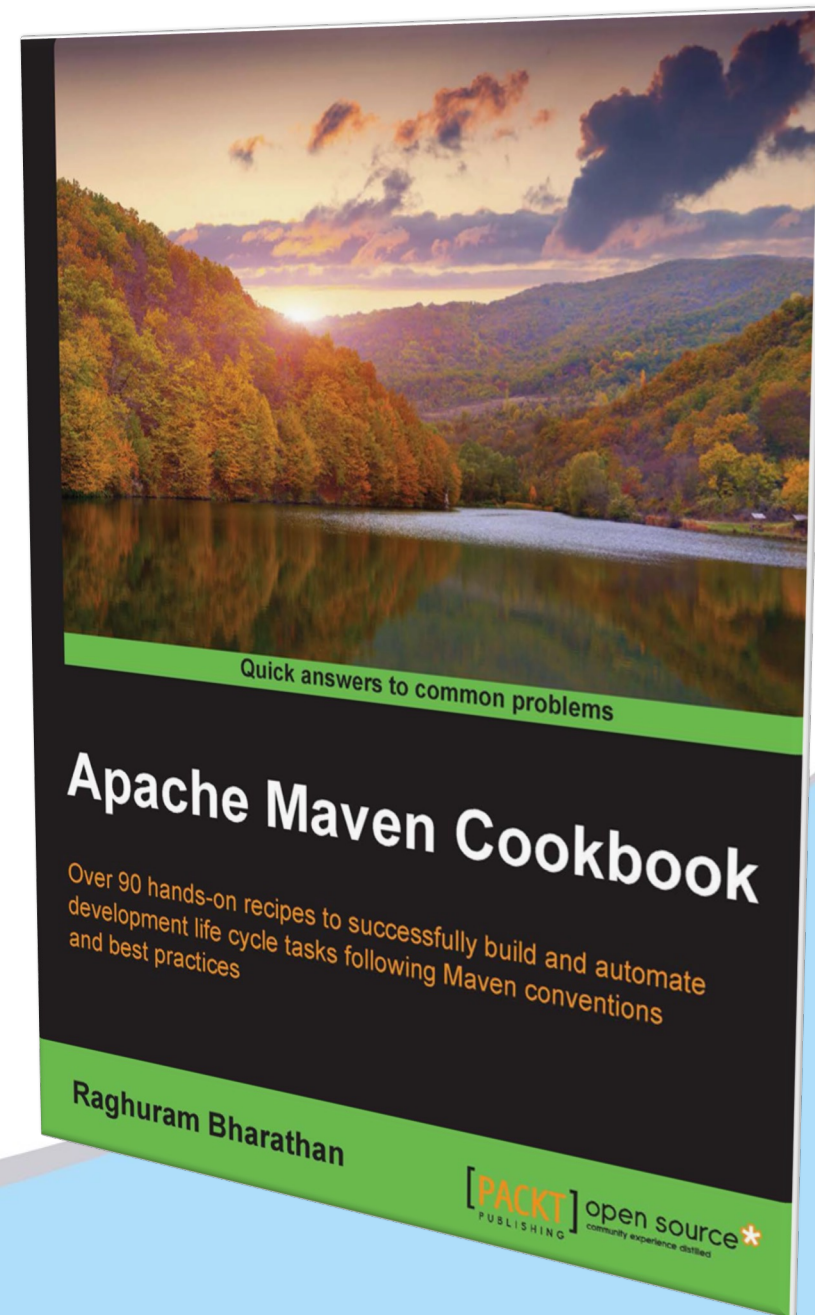
<https://maven.apache.org/background/history-of-maven.html>

- Repositório de dependências central;
- Declaração das dependências no arquivo de *build*;
- Dependências transitivas.

MVN REPOSITORY

Aprofundando em Maven

- Conceitos chave:
 - *Arquétipos (archetypes);*
 - *Objetivos (goals);*
 - *Fases (phases);*
 - *Extensões (plugins).*



Começando com arquétipos

- Arquétipos (*archetypes*) são modelos (*templates*) para projetos Maven;
 - “an original pattern or model from which all other things of the same kind are made” ([Maven Guides](#));
- Podemos usar um arquétipo existente como base;
- Podemos criar arquétipos adequados à nossa organização para padronizar nossos projetos;
- Traz as melhores práticas do Maven.

Criar um projeto Maven



- Use o arquétipo `maven-archetype-quickstart`:

Plugin

Um dos objetivos predefinidos neste *plugin*.

`mvn archetype:generate`

`-DgroupId=br.ufes.inf.pi1`

Pode mudar se quiser

`-DartifactId=mvnapp`

`-DarchetypeArtifactId=maven-archetype-quickstart`

Arquétipo

`-DarchetypeVersion=1.4`

`-DinteractiveMode=false`

Crie o projeto sem perguntar mais nada.

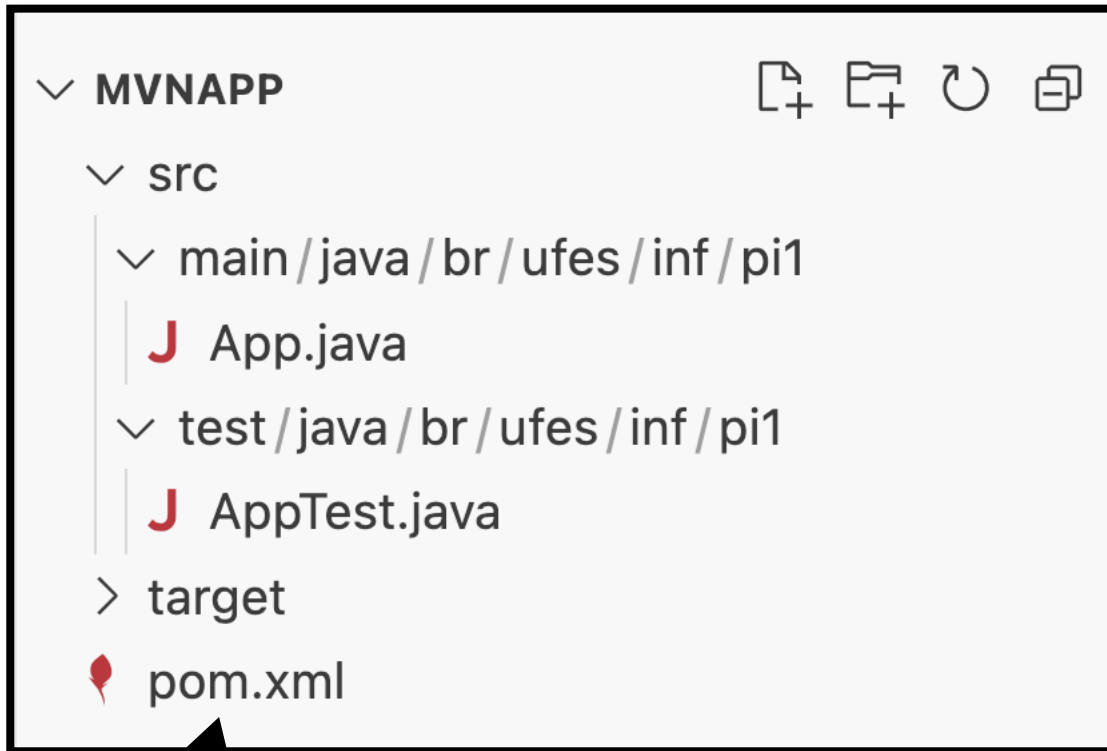
Como funciona o comando mvn

- A instalação base do Maven tem funções mínimas;
- Tarefas são geralmente feitas por *plugins*, dos quais o Maven faz *download* de um repositório padrão;
- A execução destas tarefas pode ser parametrizada;
- *Plugins* podem depender de outros e o Maven resolve e faz o *download* de tudo que precisa;
- *Downloads* são armazenados em um repositório local (~/.m2) que serve de cache.

As tarefas executadas são chamadas pelo Maven de objetivos, porém objetivos são satisfeitos enquanto tarefas são executadas. Vamos ignorar essa questão semântica daqui pra frente.



Estrutura básica de um projeto Maven



Arquivo
de *build*

Padrões

Introduction to the Standard Directory Layout

Having a common directory layout allows users to find files in a Maven project. The advantages are analogous to those of a standard file system.

The next section documents the directory layout that all Maven projects conform to this structure as much as possible. Here is a brief descriptor.

src/main/java	Application/Library sources
src/main/resources	Application/Library resources
src/main/filters	Resource filter files
src/main/webapp	Web application sources
src/test/java	Test sources
src/test/resources	Test resources
src/test/filters	Test resource filter files
src/it	Integration Tests (primarily for plugins)
src/assembly	Assembly descriptors
src/site	Site
LICENSE.txt	Project's license
NOTICE.txt	Notices and attributions required by libraries that the project depends on

a standard directory structure so you didn't have to go fishing around

<https://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html>

Fazer o *build* do projeto



- Utilize a fase package:

```
vitor@localhost mvnapp % mvn package
[INFO] Scanning for projects...
...
[INFO] Building mvnapp 1.0-SNAPSHOT
[INFO]    from pom.xml
...
[INFO] --- compiler:3.8.0:compile (default-compile) @ mvnapp ---
...
[ERROR] COMPILATION ERROR :
...
[ERROR] Source option 7 is no longer supported. Use 8 or later.
[ERROR] Target option 7 is no longer supported. Use 8 or later.
```

Alguns arquétipos estão
defasados, infelizmente...

Corrigir a versão do Java

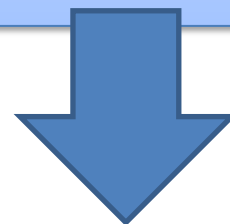


 pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

...

```
<maven.compiler.source>1.7</maven.compiler.source>
<maven.compiler.target>1.7</maven.compiler.target>
```



```
<maven.compiler.release>21</maven.compiler.release>
```

Fazer o *build* do projeto (II)



- Utilize a fase package:

O comando mvn aceita objetivos ou fases.

```
vitor@localhost mvnapp % mvn package
[INFO] Scanning for projects...
```

```
...
```

```
[INFO] Building mvnapp 1.0-SNAPSHOT
```

```
[INFO]    from pom.xml
```

```
...
```

```
[INFO] Building jar:
```

```
/Users/vitor/Workspaces/Live/mvnapp/target/mvnapp-1.0-SNAPSHOT.jar
```

```
[INFO] -----
```

```
[INFO] BUILD SUCCESS
```

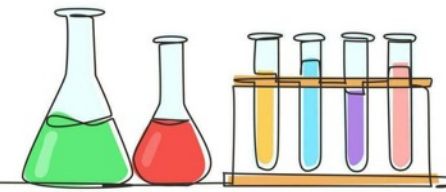
```
[INFO] -----
```

```
[INFO] Total time: 1.943 s
```

```
2024-04-01T10:27:22-03:00
```

Que objetivos de que *plugins* foram “executados” para esta fase?

Executar o projeto



- Inclua o JAR no *classpath* e execute a classe principal:

```
vitor@localhost mvnapp % java -cp target/mvnapp-1.0-SNAPSHOT.jar  
br.ufes.inf.pi1.App  
Hello World!
```

Limpar o projeto



- Executando uma fase de outro ciclo de vida:

```
vitor@localhost mvnapp % mvn clean
```

```
[INFO] Scanning for projects...
```

```
[INFO]
```

```
[INFO] -----< br.ufes.inf.pi1:mvnapp >-----
```

```
[INFO] Building mvnapp 1.0-SNAPSHOT
```

```
[INFO] from pom.xml
```

```
[INFO] -----[ jar ]-----
```

```
[INFO]
```

```
[INFO] --- clean:3.1.0:clean (default-clean) @ mvnapp ---
```

```
[INFO] Deleting /Users/vitor/Workspaces/Live/mvnapp/target
```

```
[INFO] -----
```

```
[INFO] BUILD SUCCESS
```

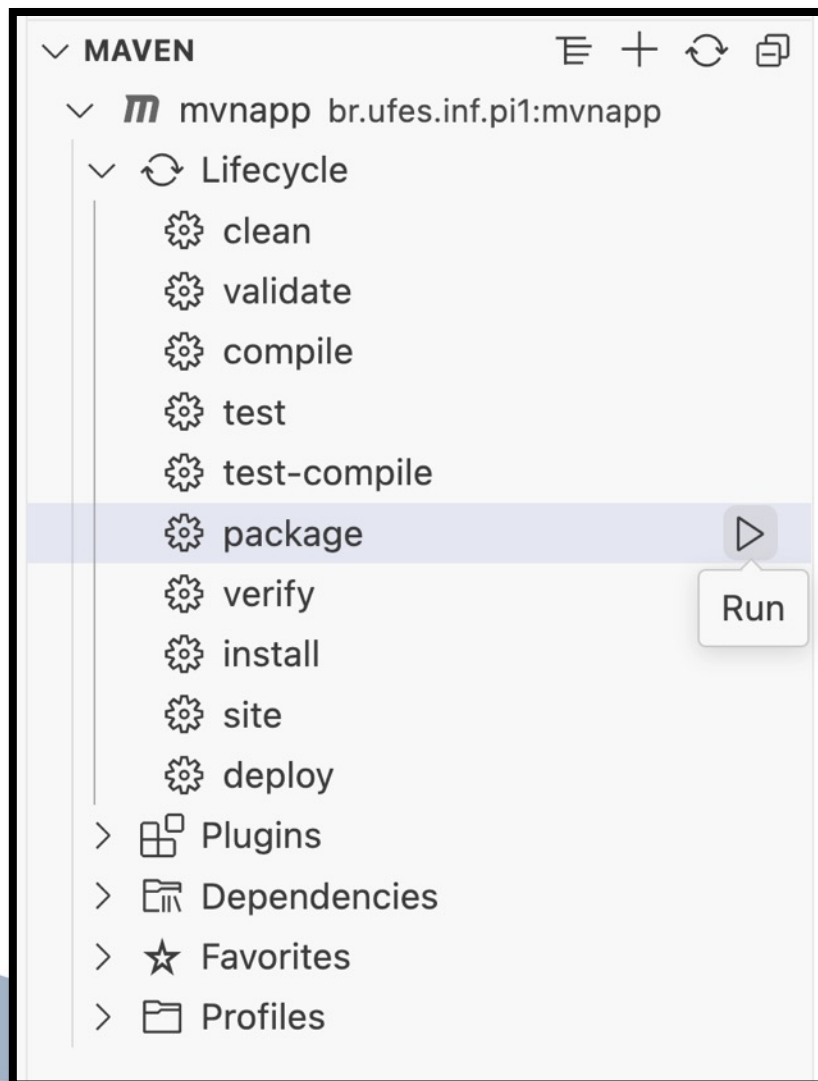
```
[INFO] -----
```

```
[INFO] Total time: 0.670 s
```

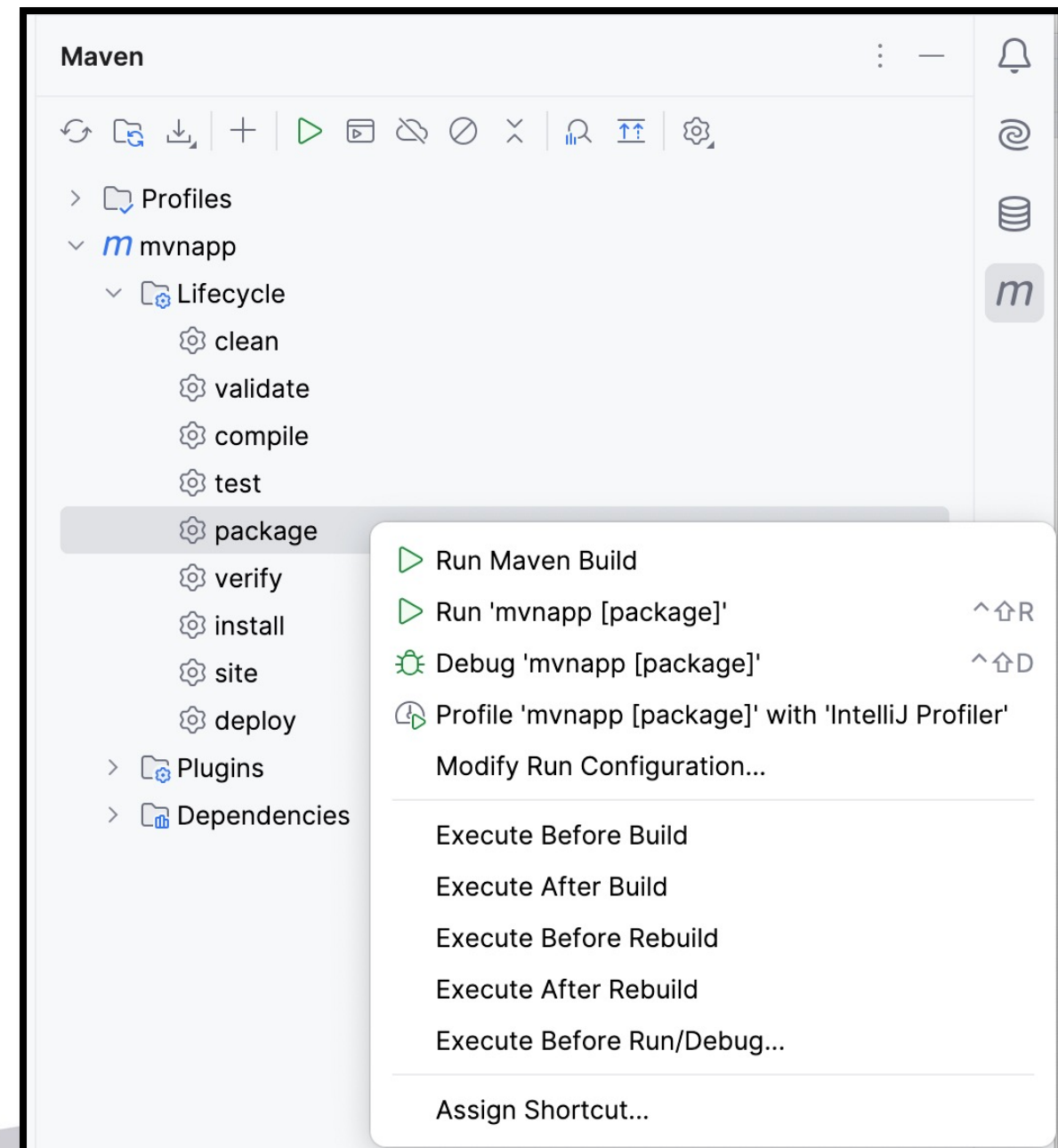
```
[INFO] Finished at: 2024-04-01T15:22:13-03:00
```

```
[INFO] -----
```

Suporte de IDEs



Visua Studio Code



IntelliJ IDEA

O que é gerado no *build*

Tudo fica dentro de target para diferenciar de conteúdo do usuário.
(Facilita o .gitignore)

✓ target

> classes

Classes compiladas e recursos copiados.

> generated-sources

> generated-test-sources

Código-fonte gerado, caso algum *plugin* o faça.

> maven-archiver

> maven-status

Informações usadas pelo Maven durante o *build*.

> surefire-reports

Relatórios de teste.

> test-classes

Classes de teste compiladas e recursos de teste copiados.

■ mvnapp-1.0-SNAPSHOT.jar

O sistema empacotado.

Fases e ciclos de vida

- O Maven define 3 ciclos de vida: *default*, *clean* e *site*;
- Cada ciclo define uma sequência de fases;
- Quando uma fase é indicada no comando mvn, o Maven executa todas as fases do ciclo até aquela.

mvn package executa até aqui

<https://maven.apache.org/ref/3.9.6/maven-core/lifecycles.html>

default Lifecycle

default lifecycle is defined **without** any associated plugin. Plugin bindings for this lifecycle are **defined separately for every packaging**:

```

1. <phases>
2.   <phase>validate</phase>
3.   <phase>initialize</phase>
4.   <phase>generate-sources</phase>
5.   <phase>process-sources</phase>
6.   <phase>generate-resources</phase>
7.   <phase>process-resources</phase>
8.   <phase>compile</phase>
9.   <phase>process-classes</phase>
10.  <phase>generate-test-sources</phase>
11.  <phase>process-test-sources</phase>
12.  <phase>generate-test-resources</phase>
13.  <phase>process-test-resources</phase>
14.  <phase>test-compile</phase>
15.  <phase>process-test-classes</phase>
16.  <phase>test</phase>
17.  <phase>prepare-package</phase>
18.  <phase>package</phase>
19.  <phase>pre-integration-test</phase>
20.  <phase>integration-test</phase>
21.  <phase>post-integration-test</phase>
22.  <phase>verify</phase>
23.  <phase>install</phase>
24.  <phase>deploy</phase>
25. </phases>
  
```


Ciclos de vida, fases e objetivos

- Um ciclo de vida define uma forma clara de construir e distribuir artefatos (fazer o *build*);
- Fases são os estágios de um ciclo de vida;
- Um ou mais objetivos (de *plugins*) podem ser “executados” em cada fase;
- O comando `mvn` pode especificar uma fase. O Maven infere a qual ciclo de vida ela pertence.

As fases mais importantes

- Ciclo de vida `clean`:
 - *`clean`: remove todos os arquivos criados pelo build.*
- Ciclo de vida `site`:
 - *`site`: gera a documentação do projeto para publicação.*
- Ciclo de vida `default`:
 - *`validate`: verifica se os dados do projeto estão corretos;*
 - *`process-resources`: copia os recursos para o destino;*
 - *`compile`: compila o código-fonte;*
 - *`test`: executa os testes unitários;*

As fases mais importantes (II)

- Ciclo de vida default (continua):
 - *package*: empacota o código compilado e recursos copiados em seu formato de distribuição;
 - *integration-test*: executa os testes de integração;
 - *verify*: verifica que o pacote gerado é válido;
 - *install*: instala o pacote no repositório local;
 - *deploy*: instala o pacote em um repositório configurado para distribuição ao público.

Fases, plugins e objetivos

Fase	Plugin	Objetivo
clean	Maven Clean plugin	clean
site	Maven Site plugin	site
process-resources	Maven Resources plugin	resource
compile	Maven Compiler plugin	compile
test	Maven Surefire plugin	test
package	Depende do empacotamento. Ex.: Maven JAR plugin	jar (neste caso)
install	Maven Install plugin	install
deploy	Maven Deploy plugin	deploy

O Project Object Model (POM)

 pom.xml



so you could look in one place for everything that pertained to the project

Cabeçalho
XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project
  xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

← Versão do POM

```
<groupId>br.ufes.inf.pi1</groupId>
```

```
<artifactId>mvnapp</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
```

Versão do software que estamos construindo.
Snapshot ≠ Release.

```
<packaging>jar</packaging>
```

...

Forma de empacotamento. Tipicamente: jar, war, zip ou pom. JAR é padrão, por isso não é incluído no arquétipo (adicionei manualmente).

Identificação
do projeto
junto ao
Maven

O Project Object Model (POM) (II)

Propriedades
do projeto

```

...
<name>mvnapp</name>
<!-- FIXME change it to the project's website -->
<url>http://www.example.com</url>

<properties>
  <maven.compiler.release>21</maven.compiler.release>
</properties>

<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.11</version>
    <scope>test</scope>
  </dependency>
</dependencies>
...

```

Nome do software
(por extenso) e site.

Declaração de dependências: outros
projetos dos quais este depende.

Escopo da dependência (ex.:
somente na fase de testes).

O Project Object Model (POM) (III)

```

...
<build>
  <pluginManagement>
    <plugins>
      <plugin>
        <artifactId>maven-clean-plugin</artifactId>
        <version>3.1.0</version>
      </plugin>
      ...
    </plugins>
  </pluginManagement>
</build>
</project>

```

Plugins a serem
utilizados no build.

A seção build pode trazer outras informações
sobre a construção do projeto, ex.: estruturas
de diretório.

Outro exemplo: uma WebApp

 pom.xml

```

...
<groupId>br.ufes.inf.labes</groupId>
<artifactId>marvin</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>war</packaging>

<name>marvin</name>
<url>https://gitlab.labes.inf.ufes.br/labes/marvin/</url>

<properties>
  <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
  <maven.compiler.release>21</maven.compiler.release>
  <failOnMissingWebXml>false</failOnMissingWebXml>
</properties>
...

```

Outro exemplo: uma WebApp (II)

```

...
<dependencies>
  <dependency>
    <groupId>jakarta.platform</groupId>
    <artifactId>jakarta.jakartaee-web-api</artifactId>
    <version>9.1.0</version>
  </dependency>
  <dependency>
    <groupId>org.primefaces</groupId>
    <artifactId>primefaces</artifactId>
    <version>12.0.0</version>
    <classifier>jakarta</classifier>
  </dependency>
  <dependency>
    <groupId>br.ufes.inf.labes</groupId>
    <artifactId>jbutler</artifactId>
    <version>2.1.2</version>
  </dependency>

```

Outro exemplo: uma WebApp (III)

 pom.xml

```
...
<build>
  <finalName>marvin</finalName>
  <pluginManagement>
    <plugins>
      ...
      <plugin>
        <groupId>org.wildfly.plugins</groupId>
        <artifactId>wildfly-maven-plugin</artifactId>
        <version>4.1.0.Final</version>
      </plugin>
    </plugins>
  </pluginManagement>
</build>
</project>
```

<https://gitlab.labes.inf.ufes.br/marvin/marvin>

Outras características dos POMs

- Podem definir relação de pai-filho em projetos multi-módulos;
 - *Projetos filhos podem herdar propriedades dos POMs pais.*
- Valores definidos em `<properties>` podem ser usados com sintaxe `${chave}` em outros pontos.

Referência completa:
<https://maven.apache.org/ref/3.9.6/maven-model/maven.html>

Executar o projeto (II)



- Configure o plugin JAR do Maven:

 pom.xml

```
<plugin>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.0.2</version>
  <configuration>
    <archive>
      <manifest>
        <addClasspath>>true</addClasspath>
        <mainClass>br.ufes.inf.pi1.App</mainClass>
      </manifest>
    </archive>
  </configuration>
</plugin>
```

```
vitor@localhost mvnapp % mvn package
vitor@localhost mvnapp % java -jar target/mvnapp-1.0-SNAPSHOT.jar
Hello World!
```

Usar uma dependência



- Adicione o Commons CSV como dependência:  pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-csv</artifactId>
    <version>1.10.0</version>
  </dependency>
```

Procure commons-csv
em mvnrepository.com

- Teste a dependência, se quiser:

```
vitor@localhost mvnapp % mvn dependency:resolve
```

...

Downloading from central:

```
https://repo.maven.apache.org/maven2/org/apache/commons/commons-
csv/1.10.0/commons-csv-1.10.0.jar
```

...

Depois confira em:
~/ .m2/repository/org/apache/

Usar uma dependência (II)



- Crie um arquivo de dados em formato CSV:

 dados.csv

```
Prédio, Nome do Prédio
CT-7, Francisco José Negreiros Gomes
CT-9, Eliane Batitucci
CT-13, Ricardo de Almeida Falbo
```

- Use o Commons CSV para ler o arquivo:

 App.java

```
public static void main(String[] args) throws Exception {
    Reader in = new FileReader("dados.csv");
    Iterable<CSVRecord> records = CSVFormat.EXCEL.parse(in);
    for (CSVRecord record : records) {
        String columnOne = record.get(0);
        String columnTwo = record.get(1);
        System.out.println(columnOne + " - " + columnTwo);
    }
}
```

Usar uma dependência (III)



- Execute dentro da IDE, suporte Maven resolve:

CT-7 – Francisco José Negreiros Gomes

CT-9 – Eliane Batitucci

CT-13 – Ricardo de Almeida Falbo

- Fora da IDE há problemas de *classpath*:

```
vitor@localhost mvnapp % mvn package
```

```
...
```

```
vitor@localhost mvnapp % java -jar target/mvnapp-1.0-SNAPSHOT.jar
```

```
Exception in thread "main" java.lang.NoClassDefFoundError:  
org/apache/commons/csv/CSVFormat  
at br.ufes.inf.pi1.App.main(App.java:17)
```

IDEs tem seu valor! 😊

Usar uma dependência (IV)



- Se faz questão de rodar no terminal:

```
vitor@localhost mvnapp % java -cp
~/.m2/repository/org/apache/commons/commons-csv/1.10.0/commons-
csv-1.10.0.jar:target/mvnapp-1.0-SNAPSHOT.jar br.ufes.inf.pi1.App
Prédio - Nome do Prédio
CT-7 - Francisco José Negreiros Gomes
CT-9 - Eliane Batitucci
CT-13 - Ricardo de Almeida Falbo
```

Bonito não é... 😅

Executar o projeto (III)



- Inclusão de um novo objetivo numa fase:

 pom.xml

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-assembly-plugin</artifactId>
      <executions>
        <execution>
          <phase>package</phase>
          <goals>
            <goal>single</goal>
          </goals>
          <configuration>
            <archive>
              <manifest>
                <mainClass>br.ufes.inf.pi1.App</mainClass>
              </manifest>
            </archive>
            <descriptorRefs>
              <descriptorRef>jar-with-dependencies</descriptorRef>
            </descriptorRefs>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
```

Atenção! Não é abaixo de
<pluginManagement>!

Executar o projeto (IV)



- Reconstrua e execute o novo JAR:

 pom.xml

```
vitor@localhost mvnapp % mvn package
```

```
...
```

```
[INFO] Building jar:
```

```
/Users/vitor/Workspaces/Live/mvnapp/target/mvnapp-1.0-SNAPSHOT-  
jar-with-dependencies.jar
```

```
...
```

```
vitor@localhost mvnapp % java -jar target/mvnapp-1.0-SNAPSHOT-  
jar-with-dependencies.jar
```

Prédio – Nome do Prédio

CT-7 – Francisco José Negreiros Gomes

CT-9 – Eliane Batitucci

CT-13 – Ricardo de Almeida Falbo

O Maven resolve! 😎

Para aprofundar,
recomendo a leitura!

